

Today I am starting with Unsupervised Machine Learning
What is Hierarchical Clustering?

Agglomerative

Divisive

Explain in detail from Zero to very advanced. Make sure the explanation should be in pointers only, use very easy going and simple language such that even 15 year old student can understand. Make use of example for better understanding. Starting from the perspective of 15 year old student take the explanation to ML engineer level. Go from explanation to mathematical implementation then coding in python.



Absolutely! Let's dive **deep into Hierarchical Clustering** in a super clear, step-by-step way—from a 15-year-old level to ML engineer level, covering **concepts, math, examples, and Python code**. I'll break it down in **pointers for clarity**.

1 Hierarchical Clustering — Basic Idea (15-year-old level)

- Imagine you have a **bunch of colored marbles**: red, blue, green, yellow.
 - You want to **group similar colors together**.
 - Hierarchical clustering is like **forming a tree of clusters** where:
 - Similar items are joined together first.
 - Then groups of similar clusters are joined together.
 - Think of it like a **family tree** for data points.
 - **Key idea**: No need to decide the number of clusters in advance (like K-Means).
 - The final output is a **dendrogram** (tree diagram showing cluster merging or splitting).
-

2 Types of Hierarchical Clustering

There are **two main types**:

A. Agglomerative (Bottom-Up)

- Start: Each data point is **its own cluster**.
- Step: Keep **merging the two closest clusters** until all points are in one big cluster.
- Analogy: Think of kids in a playground forming small friend groups and gradually all groups joining into one huge group.
- Most popular method in practice.

Steps:

1. Compute distance between each pair of points.
 2. Merge the two **closest clusters**.
 3. Recompute distances between new cluster and other clusters.
 4. Repeat until one big cluster remains.
-

B. Divisive (Top-Down)

- Start: All points are in **one big cluster**.
- Step: Keep **splitting the cluster** until each point becomes its own cluster.
- Analogy: Like a big chocolate bar you keep breaking until every square is separate.

Note: Rarely used because it's computationally expensive.

3 Distance Metrics (How we measure "closeness")

- To merge or split clusters, we need **distance between points or clusters**.
- Common **point distance metrics**:
 1. **Euclidean Distance**: Straight-line distance
$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$
 2. **Manhattan Distance**: Grid-like distance
$$d = |x_2 - x_1| + |y_2 - y_1|$$
 3. **Cosine Similarity** (angles between vectors)
- Cluster distance (how to measure distance between clusters):
 1. **Single Linkage**: Distance between closest points of two clusters.
 2. **Complete Linkage**: Distance between farthest points.
 3. **Average Linkage**: Average distance between all points.
 4. **Centroid Linkage**: Distance between cluster centers.

4 Dendrogram (The Tree)

- A **dendrogram** is a **tree diagram** showing the order of cluster merging/splitting.
 - **X-axis**: Data points.
 - **Y-axis**: Distance at which clusters were merged.
 - You can **cut the tree at any height** to decide the number of clusters.
-

5 Example with Marbles (Intuition)

Imagine 5 points on a 1D line: [1, 2, 5, 6, 8]

Agglomerative Steps:

1. Start clusters: [1],[2],[5],[6],[8]
2. Merge closest: [1,2],[5],[6],[8] (distance 1)
3. Merge next closest: [1,2],[5,6],[8] (distance 1)
4. Merge next closest: [1,2],[5,6,8] (distance 2)
5. Merge final: [1,2,5,6,8] (distance 4)

Dendrogram shows how clusters merged step by step.

6 Mathematical Implementation

Let's formalize **Agglomerative Hierarchical Clustering** mathematically:

1. Start with **N points**, each as its own cluster:

$$C = \{c_1, c_2, \dots, c_N\}$$

2. Compute **distance matrix** D where D_{ij} is distance between clusters c_i and c_j .
3. Repeat until 1 cluster remains:

- Find clusters c_p and c_q with minimum D_{pq}
 - Merge them: $c_{new} = c_p \cup c_q$
 - Update distance matrix using **linkage method**:
 - **Single linkage**: $D(c_{new}, c_r) = \min(D(c_p, c_r), D(c_q, c_r))$
 - **Complete linkage**: $D(c_{new}, c_r) = \max(D(c_p, c_r), D(c_q, c_r))$
 - **Average linkage**: $D(c_{new}, c_r) = \frac{|c_p|D(c_p, c_r) + |c_q|D(c_q, c_r)}{|c_p| + |c_q|}$
-

7 Python Implementation

python

```
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster

# Sample data: 2D points
X = np.array([[1, 2],
              [1, 4],
              [1, 0],
              [4, 2],
              [4, 4],
              [4, 0]])

# Step 1: Perform Agglomerative Hierarchical Clustering
Z = linkage(X, method='ward') # 'ward' minimizes variance within clusters

# Step 2: Plot Dendrogram
plt.figure(figsize=(8, 4))
dendrogram(Z)
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Data Points')
plt.ylabel('Distance')
plt.show()

# Step 3: Form flat clusters by cutting dendrogram at height
max_d = 5 # Threshold distance
clusters = fcluster(Z, max_d, criterion='distance')
print("Cluster labels:", clusters)
```

Explanation:

- `linkage(X, method='ward')` : Computes hierarchical clustering.
- `dendrogram(Z)` : Visualizes the hierarchy.
- `fcluster` : Cuts the dendrogram at a given height to form clusters.

8 From Beginner to ML Engineer Level

Advanced concepts:

- **Ward's method:** Minimizes total variance within clusters.
- **Scaling & normalization:** Hierarchical clustering is sensitive to scale; normalize features.
- **High-dimensional data:** Distance metrics may fail; use cosine similarity or PCA.
- **Performance:** Agglomerative has $O(n^3)$ complexity → not suitable for huge datasets.
- **Applications:**
 - Customer segmentation
 - Gene expression analysis
 - Image segmentation
 - Document clustering

ML engineer tweaks:

- Precompute distance matrix to optimize.
- Choose linkage carefully depending on your dataset.
- Use `scipy` for small datasets, `sklearn`'s `AgglomerativeClustering` for larger ones.

python

```
from sklearn.cluster import AgglomerativeClustering
```

```
agglo = AgglomerativeClustering(n_clusters=2, affinity='euclidean', linkage='ward')
labels = agglo.fit_predict(X)
print("Labels:", labels)
```

 Copy code

✓ Key Takeaways

- Hierarchical clustering creates a **tree of clusters**.
- Two types: **Agglomerative (bottom-up)** & **Divisive (top-down)**.
- No need to pre-specify clusters.
- Distance metric & linkage choice **impact results**.
- Visualize using **dendrograms**.
- Works great for **small to medium datasets**.

If you want, I can also make a **full example with 2D points + interactive dendrogram + cluster coloring** like ML engineers do in real projects. It's very visual and will make it crystal clear.

Do you want me to do that next?



Answer in short

List the parameters used for evaluating the model and calculating the loss



Here's a short list of parameters/metrics used in clustering (like hierarchical) for evaluating the model and calculating "loss" or quality:

1. Internal Evaluation Metrics (don't need true labels)

- **Inertia / Within-Cluster Sum of Squares (WCSS):** Measures compactness of clusters. Lower is better.
- **Silhouette Score:** Measures how similar a point is to its own cluster vs other clusters. Range: -1 to 1.
- **Davies-Bouldin Index:** Measures average similarity between clusters. Lower is better.
- **Calinski-Harabasz Index:** Ratio of between-cluster variance to within-cluster variance. Higher is better.

2. External Evaluation Metrics (need ground truth labels)

- **Adjusted Rand Index (ARI):** Compares clustering with true labels. Range: -1 to 1.
- **Normalized Mutual Information (NMI):** Measures information shared between clustering and true labels. Range: 0 to 1.
- **Fowlkes-Mallows Index:** Geometric mean of precision and recall of clustering vs true labels.

3. Distance-based / Loss Function

- **Euclidean / Manhattan distances** used in linkage for calculating “loss” during merging.
- **Ward’s Method:** Minimizes **variance within clusters** (used as loss).

If you want, I can also **make a tiny table mapping metric → when to use → formula** for super quick reference.

Do you want me to do that?



Do you like this personality?   